EXPERIMENT 13

Write the Python program to implement the Minimax algorithm for gaming

Register Number: **192511013**

192511013

Aim

To write a Python program to implement the Minimax algorithm, which finds the optimal move of a player in a two-player zero-sum game by searching the game tree, assuming that the opponent also plays optimally.

Algorithm

Step 1: Represent the game tree as a complete binary tree of fixed depth. The values stored at the leaf nodes are the utility values of the game from the point of view of the MAX player.

Step 2: Start at the root with MAX to move.

Step 3: If the current node is a leaf, return its utility value.

Step 4: If it is a MAX node, evaluate all its children recursively and return the largest value.

Step 5: If it is a MIN node, evaluate all its children recursively and return the smallest value.

Step 6: The value returned at the root is the optimal value that MAX can guarantee; the child that gives this value is the best move.

Step 7: Print the leaf nodes evaluated, the value at each level and the optimal value of the root.

192511013

Program

```
# Leaf values of a complete binary game tree of depth 3 (8 leaves)
leaves = [3, 5, 6, 9, 1, 2, 0, -1]
DEPTH = 3
evaluated = []          # leaf nodes visited, in order

def minimax(depth, index, is_max):
    if depth == DEPTH:                      # leaf node
        evaluated.append(leaves[index])
        return leaves[index]
    left = minimax(depth + 1, index * 2, not is_max)
    right = minimax(depth + 1, index * 2 + 1, not is_max)
    return max(left, right) if is_max else min(left, right)

if __name__ == "__main__":
    print("Game tree: depth =", DEPTH, "(root = MAX, then MIN, MAX, leaves)")
    print("Leaf values:", leaves)
    best = minimax(0, 0, True)
    print("\nLeaf nodes evaluated:", len(evaluated))
    print("Order of evaluation :", evaluated)
    print("\nOptimal value of the root (MAX player) =", best)

    # value of every internal node, level by level
    print("\nValues at each level:")
    level = leaves[:]
    names = ["Level 2 (MAX)", "Level 1 (MIN)", "Level 0 (MAX, root)"]
    for name, is_max_level in zip(names, [True, False, True]):
        level = [max(level[i], level[i + 1]) if is_max_level
                 else min(level[i], level[i + 1]) for i in range(0, len(level), 2)]
        print(f"  {name:<22}: {level}")
```

192511013

Output

```
Game tree: depth = 3 (root = MAX, then MIN, MAX, leaves)
Leaf values: [3, 5, 6, 9, 1, 2, 0, -1]

Leaf nodes evaluated: 8
Order of evaluation : [3, 5, 6, 9, 1, 2, 0, -1]

Optimal value of the root (MAX player) = 5

Values at each level:
  Level 2 (MAX)         : [5, 9, 2, 0]
  Level 1 (MIN)         : [5, 0]
  Level 0 (MAX, root)   : [5]
```

192511013